

Fixing Lazy Sender Algorithm Customization, Again

Document #: P3718R0
Date: 2025-06-28
Project: Programming Language C++
Audience: LEWG Library Evolution Working Group
LWG Library Working Group
Reply-to: Eric Niebler
<eric.niebler@gmail.com>

Contents

- 1 Background: `continues_on` and `schedule_from`:
- 2 The Problems
 - 2.1 Problem 1: A mix-up
 - 2.2 Problem 2: A mis-connect
 - 2.3 Problem 3: A muddle
- 3 Back to First Principles
 - 3.1 Special Cases
 - 3.2 Sender Consumers
 - 3.3 `get_domain_override`
- 4 Summary of Proposed Changes
- 5 Implementation Experience
- 6 Future Directions
- 7 Proposed Resolution
- 8 References

§ 1 Background: `continues_on` and `schedule_from`:

`std::execution` has two customizable algorithms for transferring execution from one context to another: `continues_on` and `schedule_from`. The reason for having two is due

to the fact there are two execution contexts in play: the context we’re transitioning *from* and the one we’re transitioning *to*.

A generic execution framework cannot know how to transition between arbitrary contexts; that is an NxM problem. Instead, `std::execution` provides a way for schedulers to separately customize how to transition to and from a standard thread of execution (ToE); i.e., `std::thread` or `main`. Transitions between unrelated contexts is accomplished with a hop through a ToE. We accomplish this by providing two customization points: one for specifying any special sauce needed to transfer *from* a standard ToE, and another for the transfer back.

The `schedule_from` algorithm looks for customizations based on the domain of the destination, and the `continues_on` algorithm dispatches based on the domain of the source. A “domain” is a tag type associated with an execution context that is used to find algorithm customizations for that context. The `continues_on` algorithm is required to lower to the result of a call to `schedule_from`. In this way, every context transition gets all the special sauce it needs to get from one arbitrary context to another.

We can see this in the definitions of the `continues_on` and `schedule_from` customizations points:

Algorithm	Returns
<code>continues_on(sndr, sched)</code>	<code>transform_sender(get-domain-early(sndr), make-sender(continues_on, sched, sndr))</code>
<code>schedule_from(sched, sndr)</code>	<code>transform_sender(query-or-default(get_domain, sched, default_domain{}), make-sender(schedule_from, sched, sndr))</code>

By asking for the predecessor sender’s domain, `continues_on` uses the domain of the source to find its customization. And by asking for the scheduler’s domain, `schedule_from` uses the domain of the destination.

The final piece is the transformation, within the connect customization point, of the `continues_on` sender to the `schedule_from` sender, which is done with the `continues_on.transform_sender(Sndr, Env)` member function (see 33.9.12.4 [\[exec.continues.on#5\]](#)).

§ 2 The Problems

§ 2.1 Problem 1: A mix-up

When connect-time customization was added to `std::execution` in [P2999R3], the logic of `continues_on/schedule_from` customization accidentally got reversed: The exposition-only *get-domain-late* function, which is called from `connect`, determines the domain used to find a sender transform function. It says:

```
template<class Sndr, class Env>
constexpr auto get-domain-late(const Sndr& sndr,
                              const Env& env) noexcept;
```

14. Effects: Equivalent to:

1. If *sender-for*<`Sndr`, `continues_on_t`> is true, then

```
return Domain();
```

where `Domain` is the type of the following expression:

```
[] {
    auto [_, sch, _] = sndr;
    return query-or-default(get_domain, sch,
                           default_domain());
}();
```

[*Note 1*: The `continues_on` algorithm works in tandem with `schedule_from` ([`exec.schedule.from`]) to give scheduler authors a way to customize both how to transition onto (`continues_on`) and off of (`schedule_from`) a given execution context. Thus, `continues_on` ignores the domain of the predecessor and uses the domain of the destination scheduler to select a customization, a property that is unique to `continues_on`. That is why it is given special treatment here. — *end note*]

2. Otherwise,

```
return Domain();
```

where `Domain` is the first of the following expressions that is well-formed and whose type is not void:

- `get_domain(get_env(sndr))`
- `completion-domain<void>(sndr)`
- `get_domain(env)`
- `get_domain(get_scheduler(env))`
- `default_domain()`

Paragraph 14.1 above gets the roles of `continues_on` and `schedule_from` mixed up. They should be reversed.

§ 2.2 Problem 2: A mis-connect

All of the adaptor algorithm CPOs use the domain of the predecessor(s) to find customizations. For example, `then(sndr, fn)` returns `transform_sender(get-domain-early(sndr), make_sender(then, fn, sndr))`; i.e., the domain is pulled from `sndr`. A sender that advertizes a domain is making an assertion about where it will complete. Where the predecessor completes is where the current sender's continuation will execute.

If we look at the connect customization point at how a late customization is found, we see that before it does anything else, it transforms the input sender as follows:

```
transform_sender(decltype(get-domain-late(sndr, get_env(rcvr)))
                {}, sndr, get_env(rcvr))
```

We can see that when passed a then sender, we ask the then sender for its domain (and use the domain of the receiver's env as a fallback). That means that for then senders, connect dispatches to a customization based on the domain of the then sender itself. That is different from early customization, which used the domain of the predecessor. The inconsistency is unintentional.

For then and most other adaptors, it doesn't make any difference. The then sender completes wherever its predecessor completes, so the domain of then is the same as the domain for the predecessor. That is not the case for all algorithms, though. For `continues_on`, the domain on which it completes can be different from the domain on which its predecessor completes.

In short, for `continues_on` and friends, connect is using the wrong domain to dispatch to a customization.

§ 2.3 Problem 3: A muddle

The connect customization point uses `get-domain-late` to determine the domain to use when applying a sender transformation. Quite apart from mixing up `schedule_from` and `continues_on`, `get-domain-late` incorrectly gives precedence to the sender's domain over that of the receiver. The (flawed) reasoning was that a sender starts where its predecessor completes, which makes intuitive sense when reading a sender chain:

```
sender auto sndr = just() | continues_on(sch) | then([] {
    puts("hello world"); });
```

Reading the above code, one might naturally infer that the then sender will start on the execution context associated with `sch`.

The trouble is: that's not true.

Senders nest and so too do their receivers and operation states. After `sndr` is connected to a receiver, calling `start` on the resulting operation state is actually calling `start` on the

then sender's operation state! The actual order of events is:

1. start is called on the then sender's operation state,
2. ... which calls start on the continues_on operation state,
3. ... which calls start on the just operation state,
4. ... which calls set_value on the continues_on receiver,
5. ... which connects and starts a schedule(sch) sender,
6. ... which causes set_value to be called on then's receiver from the execution context of sch,
7. ... which finally calls set_value on the receiver used to connect sndr.

If we want to dispatch based on where a sender will start, we should not be asking the sender. A sender can only know where it will complete. The receiver knows where it will start. The receiver is an extension of the parent sender. The parent sender starts the child, and so it can pass information to the child about where start is being called from. It does so via its receiver's environment.

Therefore, *get-domain-late* is wrong to give precedence to the sender's domain.

§ 3 Back to First Principles

The `get_domain` query actually has two meanings depending on what is being queried:

- When querying a sender's attributes, `get_domain` tells where the sender will *complete*.
- When querying a receiver's environment, `get_domain` tells where the sender will *start*.

What's more, this information propagates in different directions. Information about where senders complete is passed from left-to-right (in pipeline order) while the senders are being constructed, whereas information about where senders start is passed right-to-left while the senders are being connected.

Both bits of information – where a sender will start and where it will complete – can usefully contribute to the selection of an implementation for a sender and its successor.

1. As we build a sender up, we can use information about where the predecessor will complete – known only by the predecessor – to control how the sender is constructed.
2. As we are connecting a sender and a receiver, we can use information about where the sender will start – known only by the receiver – to select an algorithm implementation.

This is a clean and orderly separation of concerns. *get-domain-early* returns the sender's domain and *get-domain-late* returns the receiver's domain.

So are we done? Well, no.

§ 3.1 Special Cases

We still want `schedule_from` and `continues_on` to have special rules so that scheduler authors can properly orchestrate the transitions from one context to another. `schedule_from(sch, sndr)` should use `sch` to find a customization, and `continues_on(sndr, sch)` should use `sndr` to find customizations, both when building the senders and when connecting them.

The `schedule_from` customization point does not use *get-domain-early*; it only looks at `sch` when looking for a sender transform, so that part is fine. But when connecting a `schedule_from` sender, if we are only looking at the receiver's domain, then we won't be using the domain of the scheduler as we should.

The `continues_on` algorithm also needs something different. *get-domain-early* does the right thing by returning the domain of the predecessor, but again if we only use the receiver's domain in `connect`, we won't be using the predecessor's domain as we should.

The special nature of these two algorithms begs for special handling at connect time. One solution would be to special-case them in *get-domain-late*. But there is another case of interest that suggests a more general solution.

§ 3.2 Sender Consumers

Consider the following code, which schedules some work on a GPU scheduler and then waits for it to complete:

```
namespace se = std::execution;
gpu_context gpu; // non-standard

se::sender auto sndr = se::schedule(gpu.get_scheduler()) |
    se::then([]{ return 42; });
auto [result] = se::sync_wait(std::move(sndr));
```

Waiting for GPU work to complete requires GPU-specific primitives. How then should `sync_wait` find such a custom implementation? The sender knows that it will complete on the GPU, so perhaps `sync_wait` should use *get-domain-early*(`sndr`) to find a customization.

But `sync_wait` knows the environment of the receiver it will use to connect the sender. It stands to reason that `sync_wait` should use `=(sndr, sync-wait-env{...})` to determine the domain to use. This becomes more obvious when we consider a possible overload of `sync_wait` that accepts an environment as a second parameter. Certainly then, when the user has given `sync_wait` an environment, it should use it to find a customization.

The trouble is that if `sync_wait` uses *get-domain-late* to find a customization, and if *get-domain-late* only asks the environment for the domain (with special-cases for `schedule_from` and `continues_on`), then it will not find the custom GPU implementation necessary.

We have a carve-out in *get-domain-late* for `schedule_from` and `continues_on` senders. It seems we *also* need a carve-out for GPU senders ... but that's absurd! If a GPU domain need a carve-out, then other domains will surely need a carve-out too. We need a generic solution.

§ 3.3 `get_domain_override`

Senders need to have a way to override the domain of the receiver. With such a mechanism, we can replace the special-case handling of `schedule_from` and `continues_on` with the generic solution. The *get-domain-late* helper would first ask the sender if it has a “late-domain override”. If so, that is the domain returned. Otherwise, it queries the receiver’s environment as per usual.

All we need is one new sender attribute query, tentatively called `get_domain_override`. The `continues_on` and `schedule_from` senders would define this attribute, `continues_on` to return the domain of the predecessor and `schedule_from` to return the domain of the scheduler. And for the GPU sender case, the GPU domain can have an early transform that wraps all senders so that they too define that attribute.

§ 4 Summary of Proposed Changes

1. Add a non-forwarding `get_domain_override` query with no default implementation.
2. Give meaning to the `get_scheduler` query by requiring that an operation be started on an execution agent associated with the scheduler from the environment of the receiver used to create the operation.
3. Tweak the definitions of *SCHED-ATTRS* and *SCHED-ENV* to avoid forwarding the `get_domain` query.
4. Simplify the definition of the exposition-only *completion-domain* helper, which no longer needs a configurable default.
5. Specify that `get_domain_override(get_env(schedule_from(sch, sndr)))` returns the domain of `sch`.
6. Specify that `get_domain_override(get_env(continues_on(sndr, sch)))` returns the domain of `sndr` (if it has one).
7. Specify that `get_domain_override(get_env(starts_on(sch, sndr)))` returns the domain of `sch`.

8. The expression `get-domain-late(sndr, env, def)` should be equivalent to:
 1. `get_domain_override(get_env(sndr))` if that expression is well-formed.
 2. Otherwise, `get_domain(env)` if that expression is well-formed.
 3. Otherwise, `get_domain(get_scheduler(env))` if that expression is well-formed.
 4. Otherwise, `def`.
9. Specify that `sync_wait` and `sync_wait_with_variant` use `get-domain-late(sndr, sync-wait-env{ }, get-domain-early(sndr))` when looking for a customization.

§ 5 Implementation Experience

The design presented here is the result of a project to reimplement the GPU scheduler for NVIDIA's [CCCL](#) library. The old GPU scheduler, which is currently still being used by [stdexec](#), uses early customization exclusively. This requires that every algorithm is reimplemented from scratch for the GPU, resulting in a large amount of code duplication. Employing late customization would result in more accurate dispatch and facilitate more code reuse.

With `std::execution`'s current customization scheme, it was impossible for connect to find the GPU customization for the `continues_on` algorithm. Pulling on that thread revealed the other problems discussed in Section 2. Solving the problems first required a deeper understanding of the separate roles senders and receivers play in selecting a domain. That deeper understanding informed the design proposed in this paper.

The newly redesigned GPU scheduler, which uses this proposed design, can be found in [this pull request](#) for the CCCL repository on GitHub, and [this other pull request](#) implements this proposed design for `stdexec`, the reference implementation.

§ 6 Future Directions

This paper revealed a need for a `sync_wait` overload that accepts an environment in addition to a sender, like:

```
template <sender Sndr, queryable Env>
auto sync_wait(Sndr&& sndr, Env&& env);
```

With such an overload, the user could specify a scheduler corresponding to the current execution context (maybe `sync_wait` is being called from the GPU!), which would in turn determine what `sync_wait` implementation gets selected.

The `env` parameter would also give callers a way to parameterize the `sync_wait` algorithm with an allocator, or a stop token, or perhaps even a different delegation scheduler.

A separate paper will propose such an overload.

§ 7 Proposed Resolution

[Editor's note: To [execution.syn], add the following:]

```
... as before ...

namespace std::execution {
    // [exec.queries], queries
    struct get_domain_t { unspecified };
    struct get_domain_override_t { unspecified };
    struct get_scheduler_t { unspecified };
    struct get_delegation_scheduler_t { unspecified };
        struct get_forward_progress_guarantee_t {
            unspecified };
    template<class CP0>
        struct get_completion_scheduler_t { unspecified
            };

    inline constexpr get_domain_t get_domain{};
        inline constexpr get_domain_override_t
        get_domain_override{};
    inline constexpr get_scheduler_t get_scheduler{};
        inline constexpr get_delegation_scheduler_t
        get_delegation_scheduler{};
    enum class forward_progress_guarantee;
        inline constexpr get_forward_progress_guarantee_t
        get_forward_progress_guarantee{};
    template<class CP0>
        constexpr get_completion_scheduler_t<CP0>
        get_completion_scheduler{};

... as before ...
```

[Editor's note: After 33.5.5 [exec.get.domain] add a new subsection [exec.get.domain.override] as follows:]

[33.5.?] execution::get_domain_override [exec.get.domain.override]

- 1 get_domain_override asks a queryable object for the domain tag to use in connect and get_completion_signatures to find a sender transformation.
- 2 The name get_domain_override denotes a query object. For a subexpression env, get_domain_override(env) is expression-equivalent to MANDATE-NOTROW(AS-CONST(env).query(get_domain_override)).

[Editor's note: Change 33.5.6 [exec.get.scheduler] as follows:]

- 1 get_scheduler asks a queryable object for its associated scheduler.

- 2 The name `get_scheduler` denotes a query object. For a subexpression `env`, `get_scheduler(env)` is expression-equivalent to `MANDATE-NOTHROW(AS-CONST(env).query(get_scheduler))`.

Mandates: If the expression above is well-formed, its type satisfies `scheduler`.

- 3 `forwarding_query(execution::get_scheduler)` is a core constant expression and has value `true`.
- ? Given subexpressions `sndr` and `rcvr` such that `sender_to<decltype((sndr)), decltype((rcvr))>` is `true` and the expression `get_scheduler(get_env(rcvr))` is well-formed, an operation state that is the result of calling `connect(sndr, rcvr)` shall, if it is started, be started on an execution agent associated with the scheduler `get_scheduler(get_env(rcvr))`.

[Editor's note: Change 33.9.2 [exec.snd.expos#6] as follows:]

- 6 For a scheduler `sch` and queryable object `obj`, `SCHED-ATTRS(sch, _obj)` is an expression `o1` whose type satisfies *queryable* such that: [Editor's note: reformatted as a list.]
- (6.1) — `o1.query(get_completion_scheduler<Tag set_value_t>)` is an expression with the same type and value as `sch` ~~where Tag is one of set_value_t or set_stopped_t, and such that~~
- (6.2) — `o1.query(get_completion_scheduler<Tag>)` is ill-formed for Tag other than `set_value_t`,
- (6.3) — `o1.query(get_domain)` is expression-equivalent to `sch.query(get_domain)`: if that expression is well-formed, and `default_domain()` otherwise, and
- (6.4) — For a pack of subexpressions `as` and query object `Q` such that `forwarding_query(Q)` is `true`, `o1.query(Q, as...)` is expression-equivalent to `obj.query(Q, as...)`.

`SCHED-ATTRS(sch)` is expression-equivalent to `SCHED-ATTRS(sch, _env{ })`.

- ? `SCHED-ENV(sch, _obj)` is an expression `o2` whose type satisfies *queryable* such that: [Editor's note: reformatted as a list.]
- (?.1) — `o2.query(get_scheduler)` is a prvalue with the same type and value as `sch`, ~~and such that~~
- (?.2) — `o2.query(get_domain)` is expression-equivalent to `sch.query(get_domain)`: if that expression is well-formed, and `default_domain()` otherwise, and

- (?3) — For a pack of subexpressions as and query object Q such that forwarding_query(Q) is true, o2.query(Q, as...) is expression-equivalent to obj.query(Q, as...).

SCHED-ENV(sch) is expression-equivalent to SCHED-ENV(sch, env{ }).

[Editor's note: Change 33.9.2 [exec.snd.expos#8] and 33.9.2 [exec.snd.expos#9] as follows:]

```
template<class Default = default_domain, class Sndr>
    constexpr auto completion-domain(const Sndr& sndr)
        noexcept;
```

- 8 *COMPL-DOMAIN(T)* is the type of the expression `get_domain(get_completion_scheduler<T>(get_env(sndr)))`.
- 9 *Effects:* If all of the types *COMPL-DOMAIN(set_value_t)*, *COMPL-DOMAIN(set_error_t)*, and *COMPL-DOMAIN(set_stopped_t)* are ill-formed, `completion-domain<Default>(sndr)` is a default-constructed prvalue of type ~~Default~~default_domain. Otherwise, if they all share a common type (21.3.8.7 [meta.trans.other]) (ignoring those types that are ill-formed), then `completion-domain<Default>(sndr)` is a default-constructed prvalue of that type. Otherwise, `completion-domain<Default>(sndr)` is ill-formed.

[Editor's note: Change 33.9.2 [exec.snd.expos#14] as follows:]

```
template<class Sndr, class Env, class Default =
        default_domain>
    constexpr auto get-domain-late(const Sndr& sndr,
        const Env& env, Default = {}) noexcept;
```

14. Effects: Equivalent to:

~~1. If sender-for<Sndr, continues_on_t> is true, then~~

```
return
    Domain();
```

~~where Domain is the type of the following expression:~~

```
{ } {
    auto [_ , sch, _] = sndr;
    return query-or-default(get_domain, sch,
        default_domain());
};
```

~~[Note 1: The continues_on algorithm works in tandem with schedule_from ([exec.schedule.from]) to give scheduler authors a way to customize both how to transition onto (continues_on) and off of (schedule_from) a given execution context. Thus, continues_on ignores the domain of the predecessor and uses the domain of the destination scheduler to select a customization;~~

a property that is unique to `continues_on`. That is why it is given special treatment here. — *end note*

2. Otherwise,

```
return Domain();
```

where `Domain` is the type of the first of the following expressions that is well-formed ~~and whose type is not void~~:

- `get_domain_override(get_env(sndr))`
- ~~`completion-domain<void>(sndr)`~~
- `get_domain(env)`
- `get_domain(get_scheduler(env))`
- ~~`default_domain()`~~ `Default()`

[Editor's note: Insert a new paragraph after 33.9.12.3 [exec.starts.on#3] as follows:]

- ? The exposition-only class template `impls-for` is specialized for `starts_on_t` as follows:

```
namespace std::execution {
    template<>
    struct impls-for<starts_on_t> : default-impls {
        static constexpr auto get-attrs =
            [](const auto& sch, const auto& child) noexcept
            -> decltype(auto) {
                auto sch-domain = query-with-
                    default(get_domain, sch, default_domain());
                return JOIN-ENV(MAKE-ENV(get_domain_override,
                    sch-domain), FWD-ENV(get_env(child)));
            };
    };
}
```

[Editor's note: Change 33.9.12.4 [exec.continues.on#4] as follows:]

- 4 The exposition-only class template `impls-for` is specialized for `continues_on_t` as follows:

```
namespace std::execution {
    template<>
    struct impls-for<continues_on_t> : default-impls {
        static constexpr auto get-attrs =
            [](const auto& data, const auto& child)
            noexcept -> decltype(auto) {
                return JOIN-ENV(SCHED-ATTRS(data), FWD-
ENV(get_env(child)));
                return JOIN-ENV(E, SCHED-ATTRS(data,
                    get_env(child)));
            };
    };
}
```

```
};
}
```

where *E* is a queryable object such that *E*.query(get_domain_override) is expression-equivalent to get_domain(get_env(child)) if that expression is well-formed; otherwise, get_domain(get_completion_scheduler<set_value t>(get_env(child))) if that expression is well-formed; otherwise, *E*.query(get_domain_override) is ill-formed.

[Editor's note: Change 33.9.12.5 [exec.schedule.from#5] as follows:]

- 5 The member *impls-for*<schedule_from_t>::*get-attrs* is initialized with a callable object equivalent to the following lambda:

```
[](const auto& data, const auto& child) noexcept ->
decltype(auto) {
    return JOIN-ENV(SCHED-ATTRS(data), FWD-
ENV(get_env(child)));
    return JOIN-ENV(E, SCHED-ATTRS(data),
get_env(child));
}
```

where *E* is a queryable object such that *E*.query(get_domain_override) is expression-equivalent to query-with-default(get_domain, data, default_domain()).

[Editor's note: Change 33.9.12.6 [exec.on#7], as follows:]

- 7 The expression *on*.transform_env(out_sndr, env) has effects equivalent to:

```
auto&& [_, data, _] = out_sndr;
if constexpr (scheduler<decltype(data)>) {
    return JOIN-ENV(SCHED-
ENV(std::forward_like<OutSndr>(data)), FWD-
ENV(std::forward<Env>(env)));
    return SCHED-ENV(std::forward_like<OutSndr>(data),
std::forward<Env>(env));
} else {
    return std::forward<Env>(env);
}
```

[Editor's note: After 33.9.12.8 [exec.let#4], insert two new paragraphs:]

- 4 Otherwise, the expression *let-cpo*(sndr, f) is expression-equivalent to:

```
transform_sender(get-domain-early(sndr), make-
sender(let-cpo, f, sndr))
```

except that *sndr* is evaluated only once.

- ? Given a type `C` of the form `completion_signatures<Sigs...>`, let `SELECT-SIGS(C)` be a pack of those types in `Sigs` with a return type of `decayed-typeof<set-cpo>`.
- ? Given a type `Tag` and a pack `Args`, let `as-sndr2` be an alias template such that `as-sndr2<Tag(Args...)>` denotes the type `call-result-t<F, decay_t<Args>&...>`, and let `as-tuple` be an alias template such that `as-tuple<Tag(Args...)>` denotes the type `decayed-tuple<Args...>`.
- 5 The exposition-only class template `impls-for` (33.9.1 [exec.snd.general]) is specialized for `let-cpo` as follows:

```
namespace std::execution {
    template<class State, class Rcvr, class... Args>
    void let-bind(State& state, Rcvr& rcvr, Args&&...
                args);           // exposition only

    template<>
    struct impls-for<decayed-typeof<let-cpo>> :
        default-impls {
        static constexpr auto get-attrs = see below;
        static constexpr auto get-state = see below;
        static constexpr auto complete = see below;
    };
}
```

- ? The member `impls-for<decayed-typeof<let-cpo>>::get-attrs` is initialized with a callable object equivalent to the following lambda:

```
[]<class Fn, class Child>(const Fn& data, const
    Child& child) noexcept -> decltype(auto) {
    return JOIN-ENV(E, FWD-ENV(get_env(child)));
}
```

where `E` is a queryable object equivalent to

```
MAKE-ENV(get_domain, common_type_t<early-domain-of-
    t<as-sndr2<SELECT-SIGS(C)>>...>{}))
```

if that expression is well-formed, where `C` is `completion_signatures_of_t<Child>` and `early-domain-of-t<Sndr>` denotes the type `decltype(get-domain-early(declval<Sndr>()))`. Otherwise, `E` is equivalent to `env{}`.

[Editor's note: Change 33.9.12.8 [exec.let#6] as follows:]

- 6 Let `receiver2` denote the following exposition-only class template:

```
namespace std::execution {
    template<class Rcvr, class Env>
    struct receiver2 {
        ... as before ...
    };
}
```

```

Rcvr& rcvr;           // exposition only
Env env;               // exposition only
};
}

```

Invocation of the function `receiver2::get_env` returns an object `e` such that

- (6.1) — `decltype(e)` models queryable and
- (6.2) — given a query object `q`, the expression `e.query(q)` is expression-equivalent to: [\[Editor's note: Reformatted as a list \]](#)
 - (6.2.1) — `env.query(q)` if that expression is valid.
 - (6.2.2) — ~~Otherwise, `e.query(q)` is expression-equivalent to `get_env(rcvr).query(q)` if that expression is valid and the decayed type of `q` is neither `get_scheduler_t` nor `get_domain_t`.~~
 - (6.2.3) — Otherwise, `e.query(q)` is ill-formed.

[\[Editor's note: Replace 33.9.12.8 \[exec.let#8\] and 33.9.12.8 \[exec.let#9\] as shown below: \]](#)

- ~~8 Let `Sigs` be a pack of the arguments to the completion_signatures specialization named by `completion_signatures_of_t<child_type<Sndr>, env_of_t<Rcvr>>`. Let `LetSigs` be a pack of those types in `Sigs` with a return type of decayed_typeof<set_cpo>. Let `as_tuple` be an alias template such that `as_tuple<Tag(Args...)>` denotes the type `decayed_tuple<Args...>`. Then `args_variant_t` denotes the type variant<monostate, `as_tuple<LetSigs>...`> except with duplicate types removed.~~
- ~~9 Given a type `Tag` and a pack `Args`, let `as_sndr2` be an alias template such that `as_sndr2<Tag(Args...)>` denotes the type `call_result_t<Fn, decay_t<Args>&...>`. Then `ops2_variant_t` denotes the type variant<monostate, `connect_result_t<as_sndr2<LetSigs>, receiver2<Rcvr, Env>>...`> except with duplicate types removed.~~
- ? Let `C` be the type named by `completion_signatures_of_t<child_type<Sndr>, env_of_t<Rcvr>>`. Then `args_variant_t` denotes the type variant<monostate, `as_tuple<SELECT-SIGS(C)>...`> except with duplicate types removed, and `ops2_variant_t` denotes the type variant<monostate, `connect_result_t<as_sndr2<SELECT-SIGS(C)>, receiver2<Rcvr, Env>>...`> except with duplicate types removed.

[\[Editor's note: Change 33.9.13.1 \[exec.sync.wait#4\] as follows: \]](#)

- 4 The name `this_thread::sync_wait` denotes a customization point object. For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. If `sender_in<Sndr, sync_wait_env>` is false, the expression `this_thread::`

`sync_wait(sndr)` is ill-formed. Otherwise, it is expression-equivalent to the following, except that `sndr` is evaluated only once:

```
apply_sender(get-domain-earlylate(sndr, sync_wait-  
env{}), get-domain-early(sndr)), sync_wait,  
sndr)
```

Mandates:

- (4.1) — The type `sync_wait-result-type<Sndr>` is well-formed.
- (4.2) — `same_as<decltype(e), sync_wait-result-type<Sndr>>` is true, where `e` is the `apply_sender` expression above.

[Editor's note: Change 33.9.13.2 [exec.sync.wait.var#1] as follows:]

- 1 The name `this_thread::sync_wait_with_variant` denotes a customization point object. For a subexpression `sndr`, let `Sndr` be `decltype(into_variant(sndr))`. If `sender_in<Sndr, sync_wait-env>` is false, the expression `this_thread::sync_wait_with_variant(sndr)` is ill-formed. Otherwise, it is expression-equivalent to the following, except that `sndr` is evaluated only once:

```
apply_sender(get-domain-earlylate(sndr, sync_wait-  
env{}), get-domain-early(sndr)),  
sync_wait_with_variant, sndr)
```

Mandates:

- (1.1) — The type `sync_wait-with-variant-result-type<Sndr>` is well-formed.
- (1.2) — `same_as<decltype(e), sync_wait-with-variant-result-type<Sndr>>` is true, where `e` is the `apply_sender` expression above.

§ 8 References

[P2999R3] Eric Niebler. 2023-12-13. Sender Algorithm Customization.
<https://wg21.link/p2999r3>